



Security Assessment Final Report



Polymarket Oracle Aggregator

July 2026

Prepared for Polymarket

Table of Contents

Project Summary.....	3
Project Scope.....	3
Project Overview.....	3
Protocol Overview.....	3
Assessment Methodology.....	5
Threat and Security Overview.....	6
Findings Summary.....	7
Severity Matrix.....	7
Detailed Findings.....	8
Low Severity Issues.....	9
L-01: initializeRequest never validates marketType against the event ID's arity, so a misconfiguration can permanently block resolution.....	9
L-02: Arbitrator module calls can still revert execution.....	11
Informational Severity Issues.....	13
I-01: Per-EOA report bookkeeping is redundant and guards a different invariant than the one that matters.....	13
I-02: Documented operator permissions do not match enforced permissions.....	15
I-03: Rule-manager NatSpec claims it can edit request rules, but rule writes are operator/admin-gated... 16	
I-04: updateRequestRules relies on an unenforced convention that reporter modules' updateRules never reverts.....	17
I-05: Liveness window lacks minimum and maximum bounds.....	19
I-06: EOAReporterModule accepts an empty reporter set, producing a permanently inert reporter module.....	20
Disclaimer.....	22
About Certora.....	22

Project Summary

Project Scope

Project Name	Repository Link	Commit Hash	Platform
Polymarket Oracle	https://github.com/Polymarket/polymarket-v2/	Audited commit: e807826 Fix review commit: e7df9e5	EVM (Polygon)

Project Overview

This document describes the security review of **Polymarket Oracle**. The work was undertaken from **July 8th, 2026** to **July 17th, 2026**.

The following contracts were included in scope:

- `src/oracle/OracleAggregator.sol`
- `src/oracle/modules/00ReporterModule.sol`
- `src/oracle/modules/reporters/E0AReporterModule.sol`

The team performed a manual audit of all the files in scope. During the manual audit, the Certora team discovered bugs in the code, as listed on the following page.

Protocol Overview

Polymarket's modular oracle system coordinates how prediction-market outcomes are proposed, challenged, arbitrated, and finalized before being delivered to the target market contracts. OracleAggregator is the central state machine: it registers each event's reporter, disputer, arbitrator, threshold, liveness, and finalizer configuration; tallies one vote per registered module; escalates conflicting or disputed proposals; and routes the final result according to the binary, incremental negative-risk, or atomic negative-risk market shape.

The scoped changes refine that lifecycle and introduce `OOReporterModule`, which registers requests with UMA's `OOReporter`, translates settled UMA prices into Polymarket results, and permissionlessly relays reporting and finalization to the aggregator.

`EOAReporterModule` provides an alternative reporting path in which an authorized externally owned account submits a result on behalf of the module; the aggregator, rather than the individual EOA set, enforces the security-critical one-vote-per-module rule.

The review focuses on request configuration and state transitions, module registration and vote deduplication, result-shape validation across market types, liveness and dispute handling, arbitration failure isolation, UMA request/result translation, role boundaries, pause behavior, and upgrade authorization. Particular attention is given to configurations or external-module behavior that could delay, block, or incorrectly route market resolution.

Assessment Methodology

Our assessment approach combines design level analysis with a deep review of the implementation to ensure that a protocol is secure, economically sound, and behaves as intended under realistic conditions.

At the design level, we evaluate the architecture, the economic assumptions behind the protocol, and the safety properties that should hold independently of a specific chain or environment. This process includes reviewing internal and cross protocol interactions, state transition flows, trust boundaries, and any mechanism that could be exploited to extract value, deny service, or alter core system behavior. At this stage, a focused threat modelling exercise helps identify key attack surfaces and adversarial capabilities relevant to the system. Design level issues often relate to incentive structures, governance implications, or systemic behavior that emerges under adversarial conditions.

Implementation analysis focuses on the concrete behavior of the code within the execution model of the target chain. This involves reviewing the correctness of logic, access control, state handling, arithmetic behavior, and the nuanced behaviors of the chain environment. Familiar classes of vulnerabilities such as reentrancy conditions, faulty permission checks, precision issues, or unsafe assumptions often surface at this layer. These findings require context aware reasoning that takes into account both the code and the architectural intent.

To support this analysis, the codebase is examined through repeated manual passes and supplemented by automated tools when appropriate. High-risk logic areas receive deeper scrutiny, invariants are validated against both design intent and actual implementation, and potential vulnerability leads are thoroughly investigated. Automated techniques such as static analysis, fuzzing, or symbolic execution may be used to complement manual review and provide additional insight.

Collaboration with the development team plays an important role throughout the audit. This helps confirm expected behaviors, clarify design assumptions, and ensure an accurate understanding of the protocol's intended operation. All findings are documented with clear reasoning, reproducible examples, and actionable recommendations. A follow up review is conducted to validate the applied fixes and verify that no regressions or secondary issues have been introduced.

Threat and Security Overview

Protected outcome: each configured market must resolve exactly once to a shape-valid result, either through the configured reporter/disputer threshold flow or an authorized arbitrator/admin override, without unauthorized influence or avoidable loss of liveness.

- Trusted roles: owner controls UUPS upgrades; admins control roles, pause state, and emergency resolution; operators create requests and change live request configuration; rule managers maintain product specifications. Compromise or misuse of these roles can change resolution behavior.
- Semi-trusted dependencies: registered reporter, disputer, arbitrator, target, and UMA OORepoter contracts may return malformed data, revert, consume excessive gas, become unavailable, or disagree. Module identity, vote deduplication, failure isolation, and result validation form the principal trust boundaries.
- Untrusted callers: permissionless report/finalize relayers and arbitrary external accounts must not bypass registration, thresholds, liveness windows, finalizer restrictions, pause controls, or request-to-event binding.
- Primary failure modes considered: invalid or inconsistent market configuration; duplicate or conflicting votes; premature, blocked, or repeated finalization; dispute/arbitration denial of service; malformed UMA price translation; and unauthorized upgrade or role use.
- Value flow: these contracts do not custody user tokens; their security impact is indirect but critical because reported outcomes govern downstream conditional-token redemption and market settlement.

The manual review covered the three scoped contracts and their interactions at commit `e80782600c76a3b01b0714918e762d4e354b9c48`, including initialization, configuration mutation, reporting, dispute/arbitration, finalization, pause, and upgrade paths. The principal takeaways are to constrain configuration inputs, keep external-module failures from blocking the state machine, and ensure documentation precisely matches privileged capabilities and vote semantics.

Findings Summary

The table below summarizes the findings of the review, including type and severity details.

Severity	Discovered	Confirmed	Fixed
Critical	–	–	–
High	–	–	–
Medium	–	–	–
Low	2	2	1
Informational	6	6	3
Total	8	8	4

Severity Matrix

Impact	High	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Low	Low	Medium
		Low	Medium	High
Likelihood				

Detailed Findings

ID	Title	Severity	Status
L-01	initializeRequest never validates marketType against the event ID's arity, so a misconfiguration can permanently block resolution	Low	Fixed
L-02	Arbitrator module calls can still revert execution	Low	Acknowledged
I-01	Per-EOA report bookkeeping is redundant and guards a different invariant than the one that matters	Informational	Acknowledged
I-02	Documented operator permissions do not match enforced permissions	Informational	Fixed
I-03	Rule-manager NatSpec claims it can edit request rules, but rule writes are operator/admin-gated	Informational	Fixed
I-04	updateRequestRules relies on an unenforced convention that reporter modules' updateRules never reverts	Informational	Acknowledged
I-05	Liveness window lacks minimum and maximum bounds	Informational	Acknowledged
I-06	EOAReporterModule accepts an empty reporter set, producing a permanently inert reporter module	Informational	Fixed

Low Severity Issues

L-01: `initializeRequest` never validates `marketType` against the event ID's arity, so a misconfiguration can permanently block resolution

Severity: Low	Impact: Medium	Likelihood: Low
Files: src/oracle/OracleAggregator.sol	Status: Fixed	

Description:

`initializeRequest` stores the operator-supplied `marketType` verbatim without checking it against the arity encoded in the `eventId`. Arity is baked into the event ID and is the ground truth for the event's shape – binary is `arity == 0`, neg-risk is `arity >= 2`. Since `marketType` is a separate value, the two can silently disagree, and no check prevents that.

Because `marketType` selects the resolution addressing/validation regime, a mismatch blocks resolution either way:

- **Neg-risk event (arity = N) typed BINARY:** only condition 0 is addressable – legs `1..N-1` can never resolve, so a non-leg-0 winner can never be settled.
- **Binary event (arity 0) typed neg-risk:** every aggregator resolution attempt reverts. For `INCREMENTAL_NEGRISK`, `_getRequestConfigForRequestId` requires `conditionIndex < conditionCount`; both are zero for the event-level request. For `ATOMIC_NEGRISK`, `_validateResult` requires the unsigned winner index to be less than the event's zero arity, which is impossible.

`marketType` has no setter, so this is unrecoverable on-chain short of a proxy upgrade. It is a permanent resolution-DoS rather than a fund seizure as complete-set holders can still `merge/horizontalMerge` out without resolution. But directional holders can neither merge nor redeem and are stranded at mark price.

Recommendations:

In `initializeRequest`, revert on a `marketType`/arity mismatch:

- `BINARY` $\Rightarrow$ `arity == 0`
- neg-risk types $\Rightarrow$ `arity >= 2`

Optionally also assert `targetContract` matches the module implied by `marketType`.

Customer Response:

Fixed in [PR#316](#).

Fix Review:

Fix confirmed.

L-02: Arbitrator module calls can still revert execution

Severity: Low	Impact: Medium	Likelihood: Low
Files: src/oracle/OracleAggregator.sol	Status: Acknowledged	

Description:

OracleAggregator invokes arbitrator hooks using Solidity low-level calls and intentionally ignores failures:

```
if (arbitratorModule != address(0)) {
    (bool ok,) = arbitratorModule.call(
        abi.encodeCall(
            IArbitratorModule.onArbitrationTriggered,
            (_requestId, _state.proposedResultHash)
        )
    );
    if (!ok) emit ArbitratorHookFailed(_requestId, arbitratorModule);
}
```

Although the returned **bytes** value is discarded in the Solidity source, the compiler still materializes it. The generated code executes the **CALL** with a zero-sized output buffer, then reads **returndatasize()**, allocates enough caller memory for the complete payload, and copies it with **RETURNDATACOPY**.

A malicious or malfunctioning arbitrator can therefore return or revert with an excessively large payload. After the external call finishes, the aggregator must pay for memory expansion and copying that payload. If the payload is large enough relative to the aggregator's remaining gas, this operation runs out of gas before execution reaches the **if (!ok)** branch. The entire transaction then reverts, so the failure event is not emitted and any preceding state changes are rolled back.

This affects both arbitrator callbacks:

- In `_triggerArbitration`, an oversized response from `onArbitrationTriggered` can revert the dispute or reporter-conflict transaction. The write to `ArbitrationRequested` is rolled back, defeating the intended behavior of parking the request for later admin resolution.
- In the admin branch of `resolveResult`, an oversized response from `onArbitrationResolved` can revert the emergency resolution before the final result is stored.

The arbitrator is a privileged dependency and can already determine outcomes, which limits the incremental security impact. An operator or admin can also recover by replacing the arbitrator with a non-hostile address before resolving. Nevertheless, the implementation explicitly intends these hooks to be best-effort, and unbounded return-data copying violates that guarantee.

Recommendations:

Invoke both hooks with inline assembly and do not copy or inspect their return data.

Customer Response:

Acknowledged. Arbitrators are trusted entities.

Informational Severity Issues

I-01: Per-EOA report bookkeeping is redundant and guards a different invariant than the one that matters

Files:

src/oracle/modules/reporters/EOAReporterModule.sol

Description:

`EOAReporterModule.report` deduplicates reports per authorized EOA:

```
require(!hasReported[conditionId][msg.sender], AlreadyReported())
```

However, the `OracleAggregator` tallies exactly one vote per module: `reportResult()` guards on `hasReporterVoted[_requestId][msg.sender]` (where `msg.sender` is the module) and reverts the module's second vote with `AlreadyVoted`. The per-EOA guard therefore never rejects a submission the aggregator wouldn't already reject – it is redundant for vote integrity. It is also not a storage optimization – a single `hasReported[conditionId]` flag would use no more storage and be simpler. The per-EOA keying only provides per-reporter queryability and local defense-in-depth. The concern is that the two layers enforce different invariants – the module guard is per-EOA, whereas the security-critical "one vote per module" invariant is enforced solely by the aggregator's `hasReporterVoted`.

Recommendations:

Collapse the per-EOA guard to a single `hasReported[conditionId]` flag, which honestly expresses the actual behaviour ("the first authorized EOA casts the module's one vote") and removes the misleading per-EOA voting affordance.

Alternatively, if per-EOA judgment quorum is genuinely intended, forward the reporter's identity to the aggregator so it can count distinct EOAs.

Additionally, document that the one-vote-per-module invariant lives in `OracleAggregator.hasReporterVoted`, so the module-level guard is never mistaken for the enforcement point.

Customer Response:

Acknowledged. EOReporterModule is free to define its internal reporting structure. OracleAggregator aggregates reports from multiple modules, including multiple EOReporterModule instances if required.

I-02: Documented operator permissions do not match enforced permissions

Files:

docs/auth.md
src/oracle/interfaces/IOracleAggregator.sol

Description:

`docs/auth.md` states the operator "can edit a request's modules/arbitrator/finalizer/liveness [...] but cannot change [...] resolution state." This does not hold. `resolveResult()` authorizes any `msg.sender == cfg.arbitratorModule` and writes `status = Resolved`, with no `ArbitrationRequested` precondition. Since `setArbitratorModule()` lets an operator point that slot at an address it controls, the operator can transitively resolve any market. The doc's capability table lists `resolveResult()` as admin-only and `IOracleAggregator.resolveResult()` NatSpec ("after arbitration is triggered") is likewise unenforced.

Reported as **Informational only**: the project confirms `OPERATOR_ROLE` is fully trusted (Polymarket-operated keys), so there is no privilege-escalation or fund-loss impact. This is purely a documentation-correctness issue.

Recommendations:

Update `auth.md` to drop "resolution state" from the operator's "cannot change" list and do not present `resolveResult()` as admin-exclusive.

Additionally, fix the `IOracleAggregator.resolveResult()` NatSpec to match the implementation.

Customer Response:

Fixed in [PR#309](#).

Fix Review:

Fix confirmed.

I-03: Rule-manager NatSpec claims it can edit request rules, but rule writes are operator/admin-gated

Files:

src/oracle/OracleAggregator.sol

Description:

The NatSpec on `RULE_MANAGER_ROLE` (and on `addRuleManager`) in `OracleAggregator.sol` states the rule manager "can add/edit product specifications and request rules." It cannot edit rules: `updateRequestRules` is gated by `onlyOperatorOrAdmin` (`OPERATOR_ROLE | ADMIN_ROLE`) and never consults `RULE_MANAGER_ROLE`, which only authorizes `setProductSpecification` via `_authorizeMarketDataWrite`. As a result, a pure rule-manager holder cannot update request rules, while an operator can.

The same overstatement also appears on the `_authorizeMarketDataWrite` hook: its `@dev` claims it "Restricts product-spec and rule writes to rule manager or admin role holders," but it gates only `setProductSpecification` — rule writes never pass through it.

Recommendations:

Correct the `RULE_MANAGER_ROLE`, `addRuleManager`, and `_authorizeMarketDataWrite` NatSpec to state that the rule manager governs product specifications only; request rules are written by operator/admin via `updateRequestRules`.

Customer Response:

Fixed in [PR#313](#).

Fix Review:

Fix confirmed.

I-04: updateRequestRules relies on an unenforced convention that reporter modules' updateRules never reverts

Files:

src/oracle/OracleAggregator.sol
src/oracle/interfaces/IReporterModule.sol

Description:

`updateRequestRules` iterates over every registered reporter module and calls `IReporterModule.updateRules(...)` synchronously with no `try/catch`. A revert in any single module would revert the whole call, rolling back the `_pushRule` storage write and leaving every other module unnotified. So rule updates depend on an implicit, unenforced invariant that `updateRules` never reverts.

That invariant holds for all current reporter modules. There is therefore no reachable path to the issue on the current code, which is why this is reported informational only.

The concern is specific to the modular design. Reporter modules are a first-class extension point (`IReporterModule` + `addReporterModules`), yet the "must not revert" contract is neither documented on the interface nor enforced at the call site. A future reporter module whose `updateRules` can revert – for example one that adds a pause guard, a `require`, or a synchronous external call without an internal catch – would block rules propagation for every request it is registered on. The failure would be observable and recoverable (an operator can `removeReporterModules` the offending module), and only a fully trusted operator can register modules, which bounds the impact.

Recommendations:

Remove the reliance on the implicit convention, in either or both forms:

1. Document on `IReporterModule.updateRules` that implementations must not revert (best-effort; surface failures via events), and/or
2. Wrap the per-module `updateRules` call in `updateRequestRules` in a `try/catch` that emits a per-module failure event, isolating a misbehaving module instead of letting it block the batch.

Option 2 mirrors the best-effort pattern `OOReporterModule` already applies internally.



Customer Response:

Acknowledged.

I-05: Liveness window lacks minimum and maximum bounds

Files:

src/oracle/OracleAggregator.sol

Description:

`livenessWindow` is validated only as `> 0`, with no lower or upper bound. Every request is forced to be disputable (`initializeRequest` requires `disputerThreshold >= 1` and a non-zero arbitrator), yet an operator can set `livenessWindow = 1`: the dispute window (`disputeWindowEnd = block.timestamp + cfg.livenessWindow`) closes in the next block, so `finalize()` lands before any disputer can react. This is worst for EOA/Chainlink markets, where the aggregator window is the only dispute opportunity (UMA markets retain their own prior liveness).

On the other side, the symmetric misconfiguration (a near-`uint32`-max window = ~136 years) freezes finalization.

Recommendations:

Enforce a governable `minLivenessWindow/maxLivenessWindow` band in both `initializeRequest` and `setLivenessWindow` (the setter currently only re-checks `> 0`), with the admin-settable bounds themselves constrained by hardcoded `MIN_LIVENESS_WINDOW / MAX_LIVENESS_WINDOW` constant guardrails.

Customer Response:

Acknowledged.

I-O6: EOReporterModule accepts an empty reporter set, producing a permanently inert reporter module

Files:

src/oracle/modules/reporters/EOReporterModule.sol

Description:

`OOReporterModule.initializeReporterModule` rejects an empty init payload (`if (registrations.length == 0) revert EmptyRequestRegistrations();`), but `EOReporterModule.initializeReporterModule` performs no equivalent check: it decodes an `address[]` of reporters and loops to authorize them, so an empty array is silently accepted. `initOnce` still marks the request initialized, and the module is registered having authorized nobody.

An empty EOA reporter module is permanently inert and can wedge the report path:

- It can never vote. `report` requires `authorizedReporters[eventId][msg.sender]`, which is always false, so no caller can report through it.
- It can never be repaired in place. `authorizedReporters` is written only in `initializeReporterModule` under the `initOnce` guard, and there is no post-init add/remove path – re-initialization reverts `RequestAlreadyInitialized`. The only remedy is deregistering the dead module and registering a fresh one.

The trigger is a fully trusted operator's misconfiguration, and there is no fund risk, so this is informational. The point is input-validation consistency: `OOReporterModule` already guards its empty case, so the omission in `EOReporterModule` looks like an oversight.

Recommendations:

Add a non-empty check in `EOReporterModule.initializeReporterModule` mirroring `OOReporterModule`, e.g. `require(reporters.length > 0, EmptyReporters());` (or an equivalent named error), so both reporter modules reject an empty init payload consistently.

Customer Response:

Fixed in [PR#316](#).



Fix Review:

Fix confirmed.

Disclaimer

Even though we hope this information is helpful, we provide no warranty of any kind, explicit or implied. The contents of this report should not be construed as a complete guarantee that the contract is secure in all dimensions. In no event shall Certora or any of its employees be liable for any claim, damages, or other liability, whether in an action of contract, tort, or otherwise, arising from, out of, or in connection with the results reported here.

About Certora

Certora is a Web3 security company that provides industry-leading formal verification tools and smart contract audits. Certora's flagship security product, Certora Prover, is a unique SaaS product that automatically locates even the most rare & hard-to-find bugs on your smart contracts or mathematically proves their absence. The Certora Prover plugs into your standard deployment pipeline and is widely used by developers and security researchers during audits and bug bounties.

Certora also provides architectural design reviews, where researchers analyze system design and trust boundaries early to identify structural risks and reduce complexity before implementation.

Certora conducts off-chain audits covering backend services, APIs, frontend, and mobile applications, focusing on vulnerabilities in authentication, data handling, key management, and wallet-related workflows.

Certora also offers Penetration Testing and Red Teaming to simulate real-world attacks and uncover exploitable weaknesses across infrastructure, applications, and business logic.